

Exploring the Programmability of Linux Kernel Network Stack

CSS801

*Seminar Report submitted in partial fulfillment of the requirements for the degree
of PhD*

submitted by

Preeti

(Roll No. 22D0376)

under the supervision of

Dr. Mythili Vutukuru



Department of Computer Science and Engineering

INDIAN INSTITUTE OF TECHNOLOGY BOMBAY, MUMBAI

29th November, 2023

ABSTRACT

In the ever-evolving landscape of computer networks, the Linux operating system stands as a robust and versatile platform that powers a significant portion of the world's computing infrastructure. At the heart of Linux's networking capabilities lies the Network Stack, a fundamental component responsible for managing the communication between devices over a network. Traditionally, the Linux Kernel Network Stack has been perceived as a resilient and efficient entity, finely tuned to meet the demands of diverse networking scenarios. However, with the advent of complex networking requirements and the rise of software-defined paradigms, there arises a compelling need to explore and harness the programmability inherent within the Linux Kernel Network Stack.

This seminar embarks on a comprehensive exploration of the programmability aspects of the Linux Kernel Network Stack, delving into the underlying architecture, and the contemporary challenges and opportunities presented by the dynamic networking landscape. As organizations increasingly seek to tailor their network infrastructure to specific use cases and adapt swiftly to changing requirements, understanding and unlocking the programmable features of the Linux Kernel Network Stack becomes pivotal.

Through an in-depth analysis of existing literature such as Syrup: User-Defined Scheduling Across the Stack, this seminar aims to provide insights into the programmability dimensions of the Linux Kernel Network Stack. By scrutinizing the mechanisms that govern network packet processing, routing, and protocol implementation within the kernel, we intend to unravel the potential for customization and optimization. Furthermore, we will investigate emerging technologies and methodologies, such as eBPF (extended Berkeley Packet Filter), that empower developers to inject programmable logic directly into the kernel, opening new avenues for innovation and efficiency. We also focus on specific use cases that necessitate enhanced programmability in the Linux Network Stack, explaining how to enable and implement such features. To conclude, we explore future work in designing customized patched Kernel Network Stack providing re-configurable Kernel for each and every type of user applications.

Table of Contents

List of Figures	XI
1 Introduction	1
1.1 Introduction to Linux Network Stack	1
1.2 Programmability of Linux Network Stack	2
2 Current Dynamics of the Linux Network Stack	3
2.1 Initialization	4
2.1.1 Some important data structures:	6
2.2 Top Half: NIC Data Processing	6
2.3 NAPI Processing	7
2.4 Bottom Half: Network Data Processing	9
2.5 Generic Receive Offloading (GRO)	9
2.6 Receive Packet Steering (RPS)	12
2.7 Kernel Network Protocol Stack	15
2.7.1 Network Layer Processing:	15
2.7.2 Transport Layer Processing:	18
2.8 Socket Layer Processing	21

3	Current Approaches to Programmability in the Linux Kernel	24
3.1	eBPF and XDP	25
3.2	ghOSt: Fast and Flexible User-Space Delegation of Linux Scheduling . . .	26
3.3	XRP: In-Kernel Storage Functions with eBPF	29
3.4	Syrup: User-Defined Scheduling Across the Stack	32
3.5	Summary	36
4	Use Cases Demanding Enhanced Programmability in Linux Network Stack	38
4.1	Enabling and Implementing the Enhanced Programmability	39
5	Prospective Research Directions for PhD	41
	Bibliography	43

List of Figures

2.1	High Level Path taken by Packet after arrival on NIC.	4
2.2	Softirq Subsystem Initialization Process.	6
2.3	Top-Half: NIC Data Processing	7
2.4	NAPI Processing	8
2.5	Bottom-Half: Network Data Processing	10
2.6	Generic Receive Offload (GRO) Processing	10
2.7	Packet Processing: If RPS is not enabled	13
2.8	Packet Processing: If RPS is enabled	14
2.9	IP Protocol Layer Function Call Flow - Part 1	16
2.10	IP Protocol Layer Function Call Flow - Part 2	16
2.11	TCP Protocol Layer Function Call Flow	19
2.12	UDP Protocol Layer Function Call Flow	20
2.13	Socket Layer Function Call Flow	22
3.1	eBPF Architecture	25
3.2	XDP Architecture	26
3.3	ghOSt Architecture	27
3.4	ghOSt: per-CPU Scheduling Design	28
3.5	ghOSt: Thread Scheduling code in per-CPU Scheduling Design	28

3.6	ghOSt: Centralized Scheduling Design	29
3.7	Kernel Storage Stack Layer-wise Read Request Latency	30
3.8	B+ Tree Index Lookup Process From User Space	30
3.9	XRP: A Framework for In-Kernel Storage Functions	31
3.10	XRP Architecture	31
3.11	A BPF program signature and BPF function for BPF-KV	33
3.12	RocksDB benchmark performance with 100% GET requests	33
3.13	Syrup Scheduling Workflow	35
3.14	Syrup-supported hooks across the Linux Stack	36
5.1	Selective Chain of Functions	42

Chapter 1

Introduction

1.1 Introduction to Linux Network Stack

In the intricate web of modern computing, the Linux operating system stands as a cornerstone, powering a diverse array of devices and systems. At the core of Linux's networking prowess lies the Linux Kernel Network Stack, a dynamic and pivotal component that orchestrates the flow of data in the digital realm. Understanding the intricacies of the Linux Kernel Network Stack is essential for unlocking the full potential of networking capabilities in the Linux ecosystem. The Linux Kernel Network Stack, often regarded as the nerve center of network communication, is a comprehensive framework responsible for implementing and managing the protocols that facilitate seamless data exchange between devices.

Linux network stack consists of multiple segments to process the packets such as interrupt handling in top half and bottom half, NAPI (New API) processing, Generic Receive Offloading (GRO), Receive Packet Steering (RPS), IP and TCP/UDP protocol layers. As we navigate through the layers of the Linux Kernel Network Stack in the upcoming chapter, we will unravel its core components, examining how it manages packet processing, routing, and other functionalities.

1.2 Programmability of Linux Network Stack

Conventional operating systems often constrain the performance, adaptability, and capabilities of applications due to fixed interfaces and implementations of key operating system abstractions like inter-process communication, virtual memory and network stack. The research work in 1995, Exokernel operating system architecture [1], tackled this limitation by enabling applications to directly manage physical resources at the application level. As stated in Exokernel [1], the lower the level of a primitive (basic operation or functionality), the more efficiently it can be implemented. Additionally, a lower-level primitive grants more flexibility and freedom to those implementing higher-level abstractions. Likewise, we are exploring the programmability of the Linux network stack, which is a pivotal characteristic that grants developers the ability to modify, extend, and optimize networking behaviors within the Linux operating system. This feature provides a framework for injecting custom code at crucial execution points, allowing developers to tailor network functionalities to suit specific requirements. With mechanisms like eBPF (extended Berkeley Packet Filter), developers can employ a restricted C language to create programs that run at run-time, augmenting the kernel's capabilities. This programmability fosters adaptability, efficiency, and innovation within the Linux network stack, enabling the implementation of diverse networking solutions and optimizations. This introduction sets the stage for exploring the intricacies and significance of programmability in shaping the behavior of the Linux network stack.

This seminar report initiates with an exploration of the contemporary workings of the Linux Network Stack during the reception and processing of network packets, as detailed in Chapter 2. This chapter comprehensively outlines the journey of a packet from its arrival on the NIC to its ultimate delivery on the socket. Subsequently, Chapter 3 delves into an examination of current approaches supporting programmability within the Linux Kernel, encompassing technologies like eBPF, XDP, and cutting-edge solutions such as ghOSt [2], XRP [3], and Syrup [4]. Chapter 4, focuses on specific use cases that necessitate enhanced programmability in the Linux Network Stack, explaining how to enable and implement such features. The last chapter 5 discusses research direction for further research in the area of Linux network stack programmability.

Chapter 2

Current Dynamics of the Linux Network Stack

In this chapter, we will discuss the high level path taken by a packet from arrival on NIC to socket receive buffer. Before delving into the intricacies of the Linux network stack, it is essential to grasp the concept of interrupts. Hardware devices, like the Network Interface Card (NIC), issues interrupts to processors, prompting the execution of a predetermined code known as the Interrupt Handler. The Interrupt Handler operates in two parts: the **top-half**, executing in hardirq context, acknowledges the interrupt and schedules the remaining handling code for later execution in the **bottom-half**. The bottom half, running in softirq context and voluntarily scheduled by the Linux scheduler, performs the substantial processing of the received interrupt. This division ensures that interrupts are promptly acknowledged without causing prolonged disruption to the running application.

With this understanding, let's now explore the functioning of the Linux network stack from reception of a packet on NIC up to the socket layer as shown in Fig. 2.1. The operation of the kernel network stack for handling packet reception is divided into four main parts in this figure. The first part details how a network device registers and initializes during system boot, a crucial step before packets arrive on the Network Interface Card (NIC). The second part covers the **top-half** processing of the packet, encompassing actions like copying the packet into the kernel's ring buffer, generating interrupts to notify the system about packet arrival by the NIC, and handling these interrupts by the device driver, which involves scheduling the NAPI process. Further details about NAPI are explained in the

upcoming section 2.1.

The next segment focuses on the **bottom-half** processes, including NAPI polling, Generic Receive Offloading (GRO), Receive Packet Steering (RPS), and the forwarding of packets to subsequent network protocol layers, namely IP and TCP. Finally, the protocol layer addresses packet handling at the network protocol layer, transport protocol layer, and socket layer, facilitating the delivery of data to user applications.

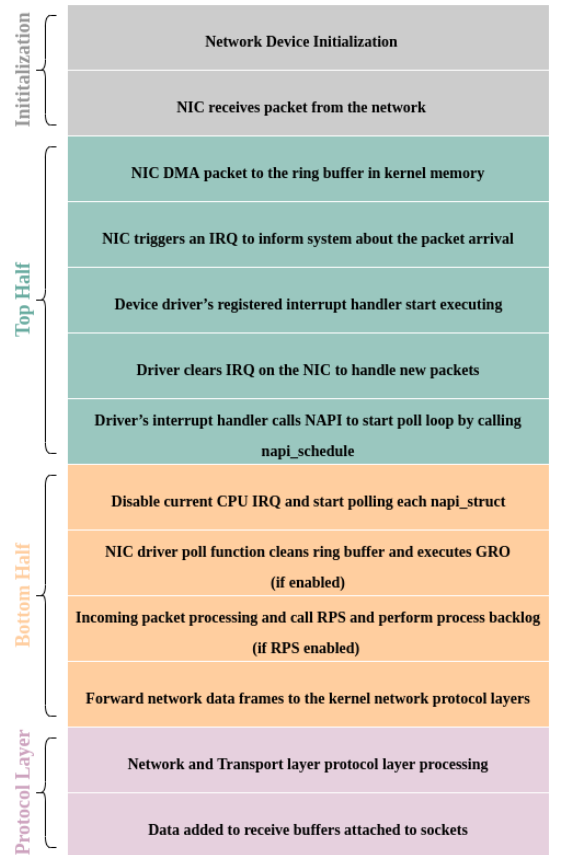


Figure 2.1: High Level Path taken by Packet after arrival on NIC.

In the upcoming sections, we will provide detailed explanations for each component depicted in the aforementioned diagram.

2.1 Initialization

The setup of a network device begins during the system boot through the device driver registration routine, as described in the online resource [5]. Taking the example of an

existing Intel device driver, 'igbe', the device driver registration routine, **'igbe'**, is invoked when the driver is loaded into the system. A device driver registration routine, **igbe_init_module()** is called whenever the driver is loaded into the system. **igbe_init_module()** invokes the **pci_register_driver()** function to register with the PCI subsystem.

During the PCI initialization process, **pci_register_driver()** utilizes the **MODULE_DEVICE_TABLE** macro when a device driver compiles, to export the **PCI_DEVICE_TABLE**, which contains IDs of PCI devices that the device driver can control. The kernel uses this table to determine which device driver to load for a specific device. **pci_register_driver()** registers a driver structure that includes function pointers and a PCI device ID table pointer. The kernel brings the PCI devices online using the corresponding driver probe function registered by the device driver. Here, the **igbe_probe()** function is registered in the **igbe_driver** structure. The **igbe_probe()** function is called by the kernel if the device is not associated with any driver. The **igbe_probe()** function enables the PCI device, requests memory ranges and I/O ports, registers the ethtool functions supported by the driver, creates the **net_device** structure representing the network device, and sets up the **net_device_ops** structure containing function pointers for various network operations. This is how a network device driver initializes the PCI subsystem during the system boot.

Now let's see how the softirq subsystem is initialized by referring Fig. 2.2. A **ksoftirqd** kernel thread is created per CPU at the time of system boot up [6]. **ksoftirqd** keeps running in loop to handle the network packets arrival interrupt and a set of data structures are created per CPU during the network device subsystem initialization. The network device (netdev) subsystem is initialized in the function **net_dev_init()**. **net_dev_init()** creates a set of struct **softnet_data** structures for each CPU on the system. These structures will hold pointers to several important things for processing network data like, list for NAPI structures (**napi_struct**) to be registered to this CPU, a backlog for data processing, processing weight, receive offload structure list, receive packet steering settings, and more as shown in Fig. 2.2. The **net_dev_init()** also registers a transmit softirq **NET_TX_SOFTIRQ** handler function **net_tx_action()** and receive softirq **NET_RX_SOFTIRQ** handler function **net_rx_action()** which will be used to process incoming or outgoing network data.

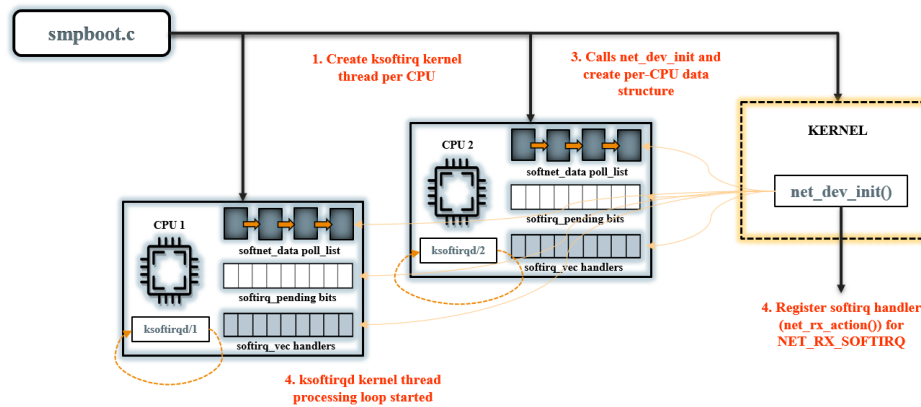


Figure 2.2: Softirq Subsystem Initialization Process.

2.1.1 Some important data structures:

softnet_data: It is created per CPU consisting of a **poll_list** of **napi_struct** structures representing each instance of the interrupting device.

napi_struct: Each interrupt vector uses one **napi_struct** to hold the state of the NAPI instance such as, network device, poll owner, napi id, GRO list, and more.

sk_buff: This data structure is used in the upcoming sections. This structure stores the network packet and is referenced by all the network stack protocol layers to store the headers, payload and metadata.

Let's start with the network packet handling within the different parts of the network stack.

2.2 Top Half: NIC Data Processing

Upon the arrival of a packet at the Network Interface Card (NIC), the card initiates the reception process, transferring the packet into an RX ring through Direct Memory Access (DMA). This ring, residing in kernel memory, serves as the destination for the incoming packet. Subsequently, the NIC interrupts the CPU to signal the presence of a new packet. The CPU, in response, transfers control to the registered NIC driver interrupt handler responsible for packet processing. The interrupt handler first clears the interrupts on the NIC, in turn, calls New API (NAPI) scheduling, initiating subsequent stages of packet

processing as shown in Fig. 2.3.

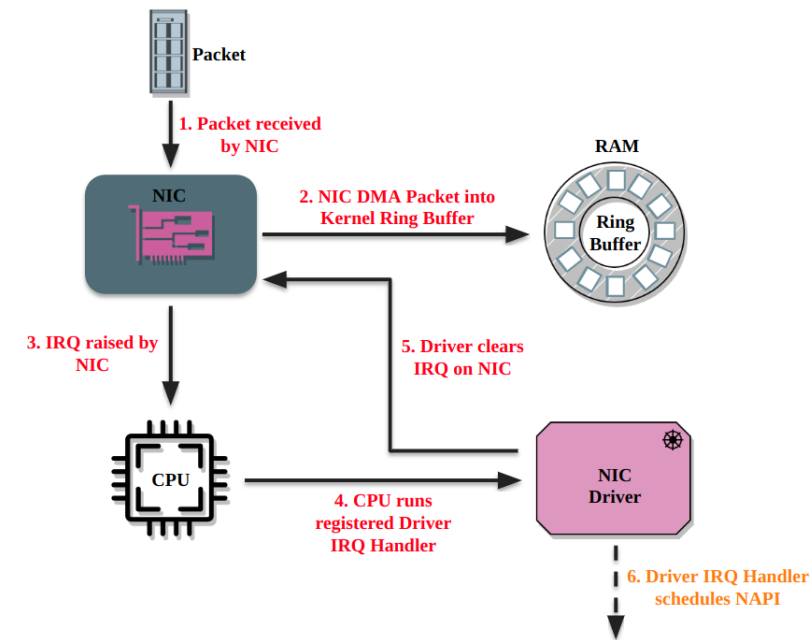


Figure 2.3: Top-Half: NIC Data Processing

2.3 NAPI Processing

When the network driver faces a high influx of packets arriving rapidly. Handling each packet individually through hardware IRQ leads to the software driver waking up frequently, causing potential overload on the system, from interrupt service processing and packet handling. To alleviate this, the New API (NAPI) was introduced. NAPI enables the polling of the IRQ line at regular intervals, allowing the retrieval of all queued data packets in the RX ring in a single operation.

Basically, NAPI serves as the interrupt handling mechanism within the Linux networking stack. The device signals the host about new events through an interrupt and the host schedules a NAPI instance to handle these events. Typically, NAPI processing occurs in the software interrupt context. It consists of two crucial components: the **napi_struct** structure and the associated poll method. Each interrupt vector needs one **napi_struct** structure to maintain the state of the NAPI instance, while the poll method serves as the driver-specific event handler. Typically, the method handles the freeing of transmitted

Tx packets and the processing of newly received packets.

Functions such as **netif_napi_add()** and **netif_napi_del()** are responsible for adding or removing a NAPI instance from the system, respectively. These instances are linked to the specified netdevice and are automatically removed when the netdevice is unregistered. When added, instances are initially in a disabled state. The management of the disabled state is handled by functions like **napi_enable()** and **napi_disable()**. A disabled NAPI cannot be scheduled, and its poll method is guaranteed not to be invoked. **napi_disable()** waits for ownership of the NAPI instance to be released.

Initiating NAPI scheduling involves the NIC driver interrupt handler invoking **napi_schedule()** function which adds the corresponding **napi_struct** structure to the **poll_list** of current CPU **softnet_data** structure, simultaneously sets the **softirq_pending** bit as shown in Fig. 2.4. Subsequently, when the current CPU's ongoing **ksoftirq** process detects the set **softirq_pending** bit, it triggers the **do_softirq()** function which, in turn, calls the registered softirq handler, the **net_rx_action()** function. **do_softirq()** determines this softirq handler function by checking **softirq_vec** handlers list of the current CPU. Ultimately, the **net_rx_action()** handler function initiates the NAPI poll loop which we will discuss in the next section.

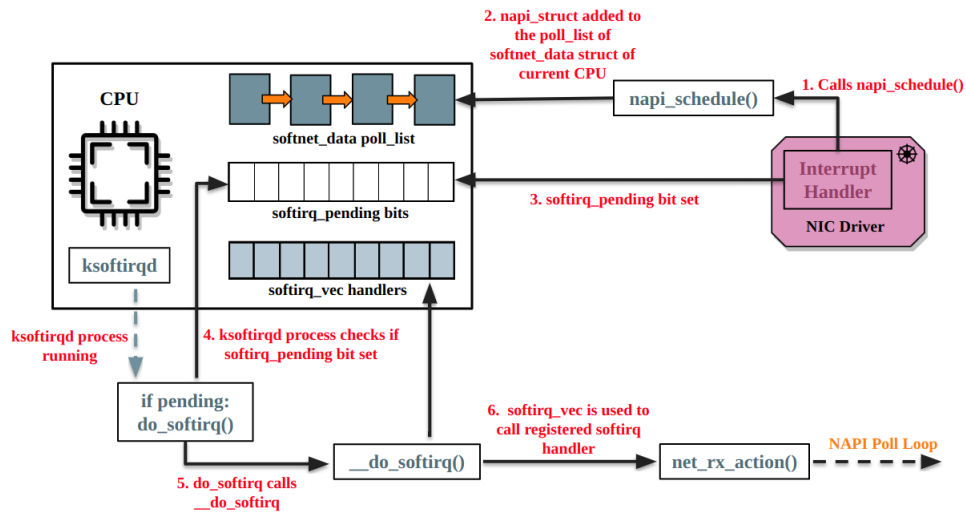


Figure 2.4: NAPI Processing

2.4 Bottom Half: Network Data Processing

The `net_rx_action()` function temporarily suspends the current CPU interrupts until all packets within the RX ring of each registered device are managed by the Soft IRQ. This function systematically polls each of the registered device and processes the RX ring of each of the device as shown in Fig. 2.5. It iterates through the `poll_list` of current CPU `softnet_data` structure. For each `napi_struct` in the `poll_list`, general `poll()` function is invoked, triggering the corresponding registered driver poll function, denoted as `xyz_poll()`, where 'xyz' represents the specific driver, such as `igb`, `igc`, etc. The driver's `xyz_poll()` function, executes the `xyz_clean_rx_irq()` function to collect packets from the RX ring buffer. Subsequently, it proceeds to the Generic Receive Offload (GRO) processing, as discussed in the upcoming section 2.5. The `net_rx_action()` constrains the execution of driver's poll function (`xyz_poll()`) by keeping track of execution time and work budget. This technique employed by the kernel safeguards against packet processing monopolizing the entire CPU. The work budget denotes the total number of packets assigned for processing across all the NAPI structures registered to the current CPU. At the end, it checks the presence of any packet awaiting processing for Receive Packet Steering (RPS) in the `input_pkt_queue`, as discussed in the upcoming section 2.6. It triggers the activation of the remote CPU, if not active, to start the processing of network data. This is achieved using Inter-Process Interrupt (IPI) called within the function `net_rps_action_and_irq_enable()` as discussed in section 2.6.

2.5 Generic Receive Offloading (GRO)

Generic Receive Offload (GRO) stands as a prevalent software-based offloading technique designed to minimize per-packet processing overheads, as described in the Linux Kernel Documentation [7] and [5]. GRO achieves this by consolidating small packets into larger ones, allowing applications to handle fewer, more substantial packets directly. Consequently, this approach diminishes the overall number of packets that need to undergo processing.

Consider a scenario where a substantial file transfer is underway, and the majority of the packets comprise fragments of data from the file. Rather than dispatching small packets individually up the stack, the incoming packets can be consolidated into a single packet

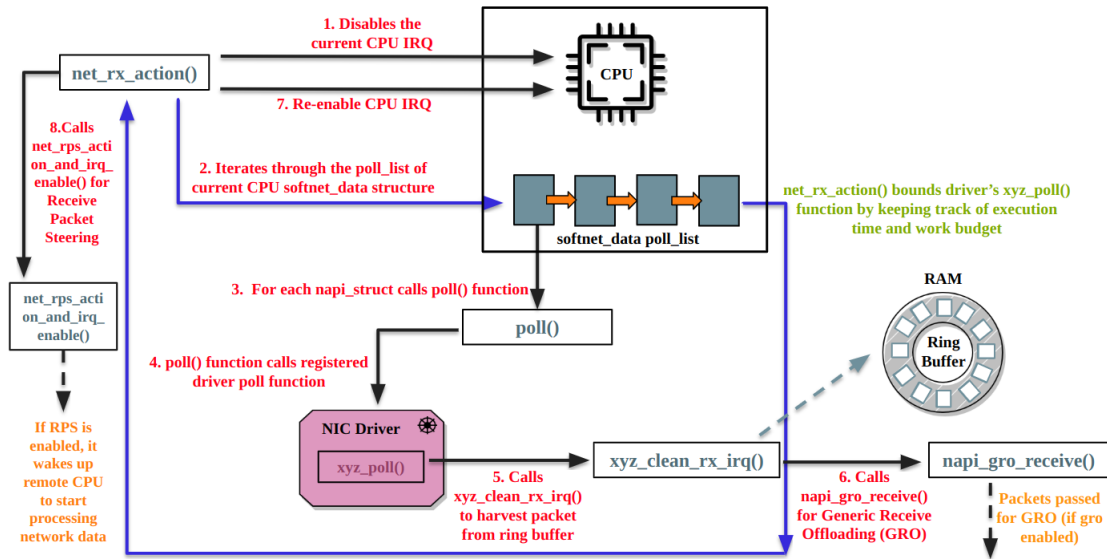


Figure 2.5: Bottom-Half: Network Data Processing

featuring a sizable payload. This consolidated packet is then forwarded up the stack, facilitating the protocol layers to handle the headers of a single packet while concurrently delivering more substantial data chunks to the user program.

Now, let's explore its functioning within the network stack.

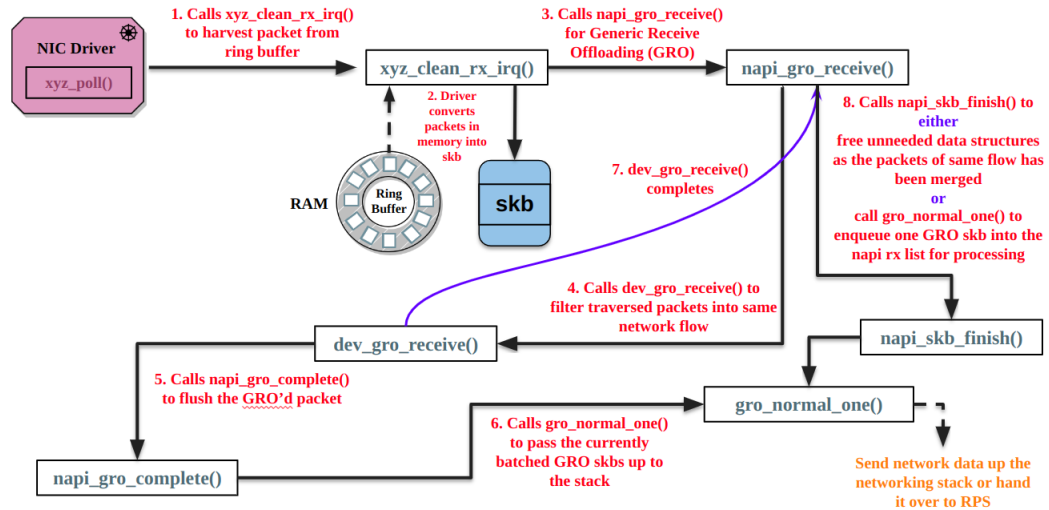


Figure 2.6: Generic Receive Offload (GRO) Processing

As we have seen in the previous section, the driver's poll function invokes **xyz_clean_rx_irq()** to harvest packets from the RX ring buffer and converts packets in memory into

skb (**sk_buff** structure) format. Then the driver poll function calls **napi_gro_receive()** function to start GRO processing and subsequently calls **dev_gro_receive()** function.

The **dev_gro_receive()** function starts by verifying the status of Generic Receive Offload (GRO) activation. In the scenario where GRO is enabled, the function traverses a list of GRO offload filters such as network header and packet type. This traversal allows higher-level protocol stacks to take action on a data segment under consideration for GRO. **dev_gro_receive()** checks the packet protocol from driver, if it matches the packet offload ethernet type. If matches successfully, then reset the network header and mac length of the **skb** and initialize the **napi_gro_cb** structure fields. Setup the GRO checksum validation and then call the **inet_gro_receive()** to process IP protocol related GRO processing of the packet. **inet_gro_receive()** further calls **tcp4_gro_receive()** function for TCP related GRO processing of packet. The purpose is to enable protocol layers to inform the network device layer whether the packet is part of a network flow currently undergoing receive offloading. This interaction allows for handling protocol-specific requirements related to GRO such as header and length changes. For instance, the TCP protocol must make decisions regarding when to acknowledge a packet being consolidated into an existing packet.

Next, **dev_gro_receive()** calls **napi_gro_complete()** to flush the GRO'd packet by further calling **inet_gro_complete()** and **tcp4_gro_complete()** for the respective protocol layers. If no error returned, the currently batched GRO_NORMAL skbs are then passed up to the stack by calling **gro_normal_one()**. If **gro_normal_one()** finds **napi_rx_count** greater than 1, then calls **gro_normal_list()** to pass the currently batched GRO_NORMAL skbs up to the stack and reset the **rx_count** to 0. Upon returning from the **dev_gro_receive()** to **napi_gro_receive()**, **napi_skb_finish()** is called to either release the unneeded data structures as the packets of the same flow has been merged or calls **gro_normal_one()** to enqueue one **skb** into the **napi RX** list for further processing as the packet is not destined to be merged with an existing GRO'd flow. **gro_normal_one()** subsequently passes the network data upwards to the networking stack or hand it over to the RPS process, as discussed in next section [2.6](#).

2.6 Receive Packet Steering (RPS)

In previous discussion, we highlighted that network device drivers register a NAPI poll function. Each instance of a NAPI poller operates within the context of a softirq, with one such softirq per CPU. And, the CPU on which, driver's IRQ handler running, triggers its softirq processing loop to handle incoming packets. That means, a single CPU manages the hardware interrupt and polling for packets to process incoming data. Certain NICs, such as the Intel I350, offer support for multiple hardware-level queues. This capability allows incoming packets to be Direct Memory Access (DMA) transferred to distinct memory regions for each queue, each managed by a separate NAPI structure. Consequently, multiple CPUs handle interrupts from the device and process the packets. This functionality is commonly known as Receive Side Scaling (RSS).

Receive Packet Steering (RPS) serves as a software implementation of RSS, as defined in the Linux Kernel Documentation [8]. Given its software nature, RPS can be enabled for any NIC, even those with only a single RX queue. However, since it operates in software, RPS comes into play after a packet has been harvested from the DMA memory region.

Now, let's proceed with the analysis of the packet flow by exploring how RPS operates within the network stack.

As shown in Fig. 2.7, **gro_normal_one()** invokes **netif_receive_skb_list_internal()** which checks for some receive time-stamping, either before or after a packet enters the backlog queue, based on the user preferences. Following time-stamping, it verifies the status of RPS. If RPS is not enabled, it directly calls the **__netif_receive_skb_core** function to pass the packets to the protocol stack by calling **deliver_skb()** function. Besides this CPU copies the data to the socket, facilitating data delivery to packet taps like XDP, through the **do_xdp_generic()** function. If RPS is enabled, it proceeds to call the **enqueue_to_backlog()** function as illustrated in Fig. 2.8.

Now let's explore the RPS processing as described in Fig. 2.8.

The **netif_receive_skb_list_internal()** function initially verifies whether the RPS is enabled. If enabled, the function proceeds to invoke the **get_rps_cpu()** function for packet steering and enqueues the skb in per-CPU backlog. The target CPU is determined from the RPS map of the receiving queue associated with the given skb, ensuring that the data gets queued to the desired CPU's backlog. Subsequently, the SKB entry is removed from

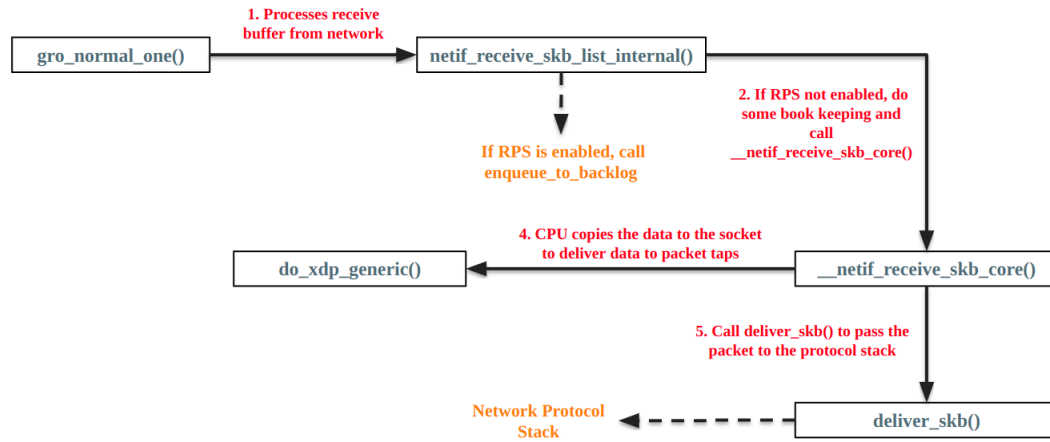


Figure 2.7: Packet Processing: If RPS is not enabled

the list, and the `enqueue_to_backlog()` function is called for further processing.

The `enqueue_to_backlog()` function first finds the pointer to the target CPU's `softnet_data` structure to obtain the pointer to the `input_pkt_queue` and then checks the `input_pkt_queue`'s length. If the queue length exceeds the maximum backlog queue length, the data is dropped. Additionally, the function checks the skb flow limit to prevent the system from managing a high rate softirq processing. Then packets are then enqueued in the CPU's `input_pkt_queue` and the `napi_schedule_rps()` is invoked to service the queue as soon as possible.

Within `napi_schedule_rps()`, the function checks if the pointer to the `softnet_data` belongs to the current CPU. If it belongs to a remote CPU, the remote CPU's `napi_struct` is added to the current CPU's `softnet_data` RPS initialization list `rps_ini_list`. If the current CPU is not actively running `net_rx_action()` and not engaged in the NAPI polling, the `NET_RX_SOFTIRQ` is raised using the `__raise_softirq_irqoff()` function. If the pointer is associated with the same CPU, the backlog NAPI is scheduled by calling the `napi_schedule()` function.

Whenever the remote CPU's kernel thread `ksoftirqd` runs and finds packets in the per-CPU backlog queue, the backlog poll function `process_backlog` is invoked to handle these packets in softirq context. Inside the `process_backlog` function, if it detects the presence of pending RPS IPIs using function `sd_has_rps_ipi_waiting()`, it triggers the `net_rps_action_and_irq_enable()` function. This function, in turn, examines the `softnet_data rps_ini_list` and dispatches pending IPI's to initiate RPS processing on remote CPUs if

finds non-empty `rps_ini_list` by calling `net_rps_send_ipi()` function.

The `net_rps_send_ipi()` function further calls the `smp_call_function_single_async()` function with specific cpu id and `call_single_data` structure passed as arguments. This function executes an asynchronous function on the specified CPU if the remote CPU is active.

After initiating the `net_rps_action_and_irq_enable()` function, the `process_backlog` function enters a loop, persisting until its allocated weight (quota of number of packet's processing) is fully utilized, or no more data remains in the backlog. In each iteration of the loop, a data segment is dequeued from the backlog queue using `__skb_dequeue()` function and forwarded to `__netif_receive_skb()` function.

In the subsequent path, upon reaching `__netif_receive_skb`, the process follows the same sequence as described earlier for the scenario when RPS is disabled. In essence, `__netif_receive_skb()` performs preliminary timestamp bookkeeping tasks before invoking `__netif_receive_skb_core()` to elevate network data to the protocol layers by invoking `deliver_skb()` and deliver packets to the packet taps by calling `do_xdp_generic()`.

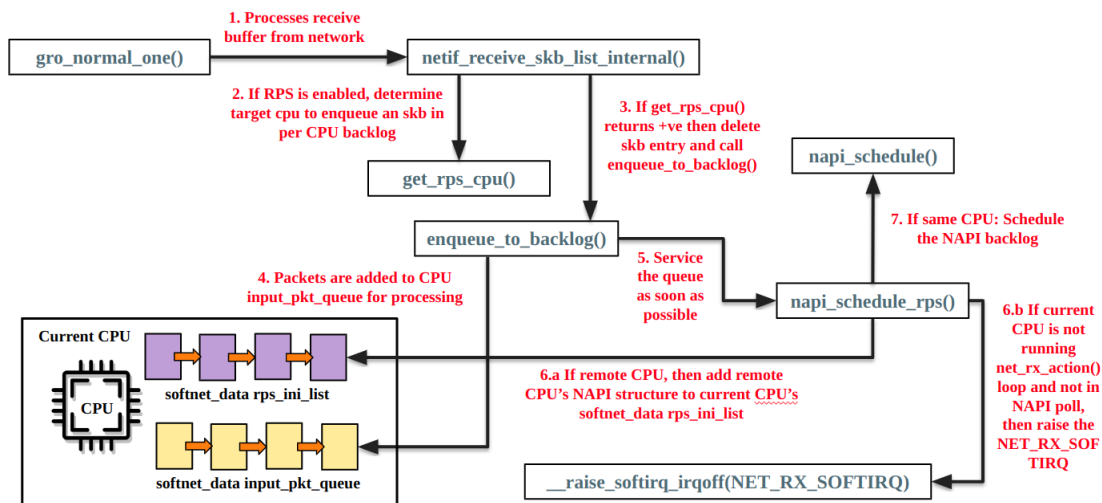


Figure 2.8: Packet Processing: If RPS is enabled

2.7 Kernel Network Protocol Stack

As discussed in the last section, `__netif_receive_skb()` function is responsible for classifying packets based on their type and directing them to the relevant registered packet handler function by calling `deliver_skb()` function. In the following sections, we will delve into the process through which packets from the net device layer traverse into the layers of the kernel's network protocol stack.

2.7.1 Network Layer Processing:

Before delving into the IP layer functioning, first let's see, how IP layer protocol registers itself into the network stack.

The Linux kernel employs the `packet_type` structure to represent different packet types within the networking stack. The IP layer protocol registers itself as a packet type through `packet_type` structure and `ptype_base` field is a numerical identifier that helps in categorizing and distinguishing different packet types.

In essence, `ptype_base` is defined as a hash table of lists, facilitating the determination of which packet type handler should be invoked for processing an incoming packet. As part of the registration process, the IP protocol layer adds itself to a specific slot in the `ptype_base` hash table, and this slot is determined using a helper function named `ptype_head`.

At the boot time, the initialization process is orchestrated by `inet_init()`, which oversees the initialization of all subsystems related to IP and layer 4 protocols. During this initialization, `inet_init()` calls `ip_init()` to initialize IP protocol at boot time. The `ip_init()` function calls `dev_add_pack()` to register the IP protocol handler function, here it is `ip_rcv()`.

As previously discussed, when `deliver_skb()` is invoked, it triggers the execution of the handler function `ip_rcv()`, corresponding to the protocol type stored in the `packet_type` structure of the packet as shown in Fig. 2.9.

`ip_rcv()` is the central function for handling the entry of a packet from net device layer into the network layer. `ip_rcv()` invokes `ip_rcv_core()` function to perform all the sanity checks.

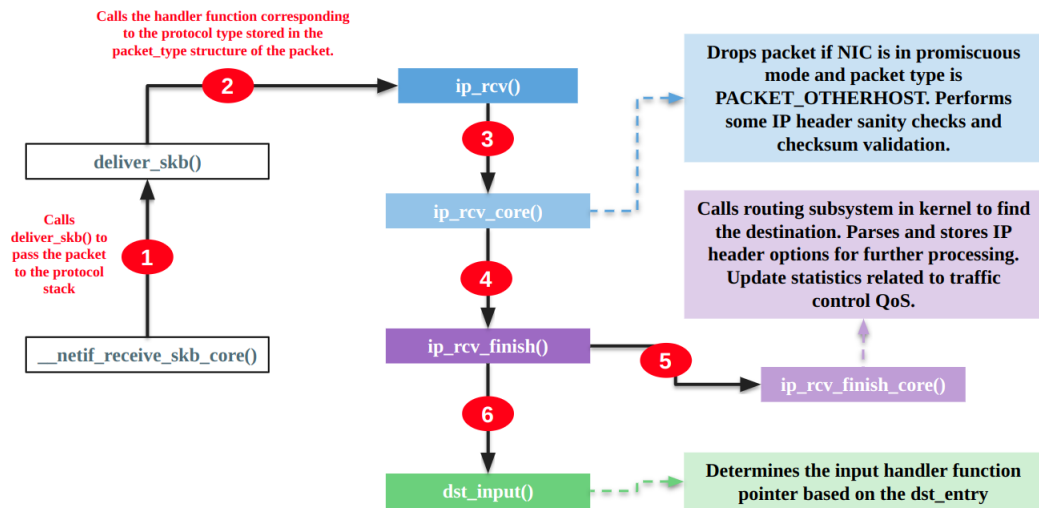


Figure 2.9: IP Protocol Layer Function Call Flow - Part 1

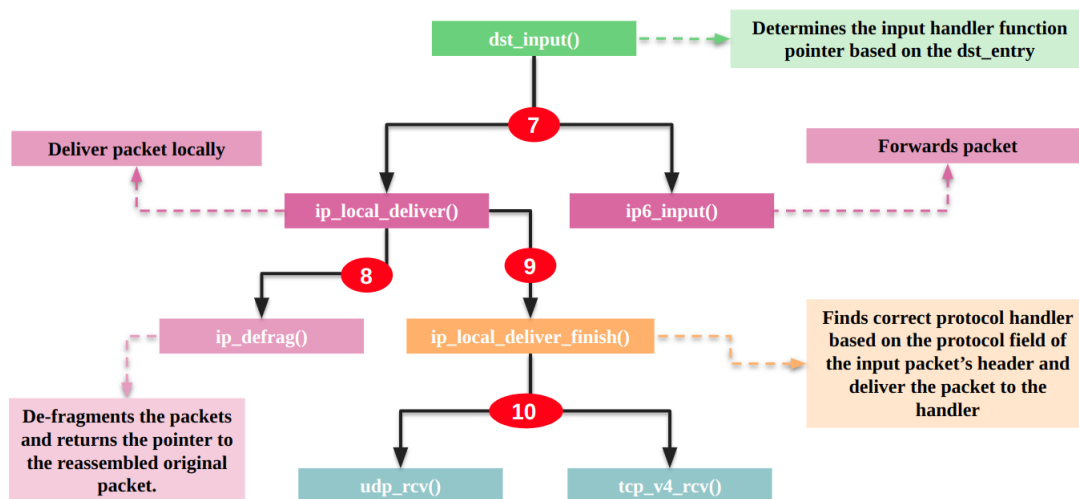


Figure 2.10: IP Protocol Layer Function Call Flow - Part 2

Within `ip_rcv_core()` function, several checks are carried out to ensure the integrity of the packet. If NIC is in promiscuous mode and if the packet type is set to **PACKET_OTHERHOST**, the packet is dropped. The function also checks whether the skb is shared with other users using `skb_share_check()` function. If sharing is detected, the function creates a separate copy of packet and pass the original packet.

The `pskb_may_pull()` function is called to ensure that the skb data points to a block of data at least as big as the IP header. Following these sanity checks, it performs header ver-

ification and checksum validation before returning back to **ip_rcv_core()**. If the packet has not been dropped at this point, it undergoes the pre-routing net filter hook, specifically **NF_INET_PRE_ROUTING** function within the **NF_HOOK**. Subsequently, the packet enters the **ip_rcv_finish()** function for further processing.

ip_rcv_finish() function performs packet's route lookup through the execution of **ip_rcv_finish_core()** and determines whether the packet should be delivered locally or forwarded by making a call to the **dst_input()** function.

Initially, **ip_rcv_finish_core()** examines the destination entry (**dst_entry**) of the skb to acquire the route to the destination. If destination entry is null, the function invokes the routing subsystem either through **early_demux()** or **ip_route_input()**, to populate the skb **dst_entry**. In the case of **early_demux()**, the skb **dst_entry** is obtained from the destination entry cached in the socket structure of the packet. On the other hand, **ip_route_input()** locates the entry based on the source and destination addresses stored in the packet structure. Following this, the function updates statistics related to traffic control Quality of Service (QoS). The **skb_cow()** is then called to create a copy of the buffer if it is shared with other users.

To interpret the IP options carried in the header, **ip_rcv_options()** is utilized. **ip_rcv_options()**, in turn, calls **ip_options_compile()** to parse the options and store them in an **ip_options** structure within the data field pointed to by the skb control buffer. Additionally, **ip_rcv_options()** function calls the **ip_options_rcv_srr()** to set the **dst_entry** and determine the device responsible for handling the packet towards the next hop in the route list. Upon completing these operations, **ip_rcv_finish_core()** returns back to **ip_rcv_finish()**.

Finally, **ip_rcv_finish()** concludes its operations by invoking the **dst_input()** function, which in turn, invokes the input handler function pointer based on the corresponding **dst_entry** structure of the skb, as determined by the routing subsystem. The handler function can be either **ip_local_deliver()** for delivering the packet on the local system or **ip6_input()** for handling different destination. Let's explore the details of local delivery function, **ip_local_deliver()**.

The **ip_local_deliver()** function is responsible for delivering IP packets to the higher protocol layers. Initially, it checks if the packet is fragmented, and if so, it calls **ip_defrag()** to handle the de-fragmentation of the packet and returns a pointer to the original re-

assembled packet. Subsequently, the packet traverses the net filter hook, specifically the **NF_INET_LOCAL_IN** function within **NF_HOOK**. Afterward, the packet enters **ip_local_deliver_finish()** function for further processing.

The **ip_local_deliver_finish()** function proceeds by identifying the relevant transport layer protocol handler based on the protocol field in the input packet's header and delivers the packet to either **tcp_v4_rcv()** or **udp_rcv()** handler function, to forward the packet towards transport layer processing. It also handles the raw IP and enforce security policies.

2.7.2 Transport Layer Processing:

Before delving into the transport layer functioning, let's explore how a transport layer protocol registers itself into the network stack.

The **net_protocol** data structure is used to define Layer 4 (L4) protocols that are registered with the kernel network stack. This structure includes handler functions that establish connections between UDP/TCP protocols and the IP protocol layer. The **inet_protos_table** encompasses all the registered L4 protocols. During the system boot time, the **inet_init()** function invokes **inet_add_protocol()** to register L4 protocols, initializing the network subsystem. In the further discussion, we will delve into the details of both the UDP handler function, **udp_rcv()**, and TCP handler function, **tcp_v4_rcv()**.

- **TCP**

The flow of TCP protocol handler function is shown in Fig. 2.10.

tcp_v4_rcv() handler function initiates by validating the packet and TCP header using **pskb_may_pull()**, which ensures the presence of a complete TCP header in the skb. Subsequently, it verifies the checksum with **skb_checksum_init()** and stores the TCP header and IP header for further processing. In the next step, the function seeks an open socket for the the incoming skb through **_inet_lookup_skb()**. **_inet_lookup_skb()** calls **skb_steal_sock()** to retrieve the socket from the **sk_buff** structure. If not found in the **sk_buff** structure, then calls **_inet_lookup()** to locate the socket based on the net device, source and destination addresses, and port numbers.

In case where **sk_state** is **TCP_NEW_SYN_RECV**, checksum verification is performed. If the **sk_state** is **TCP_LISTEN**, the socket is obtained from the

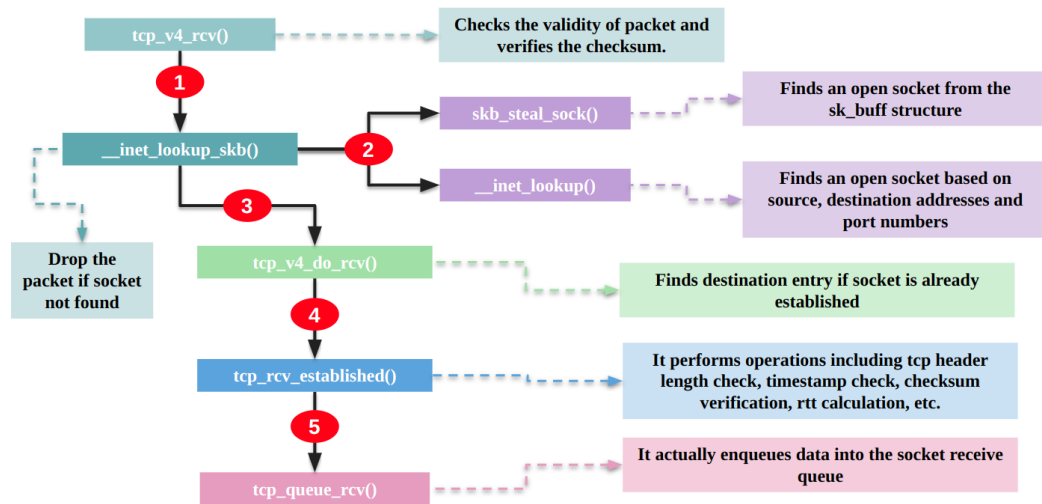


Figure 2.11: TCP Protocol Layer Function Call Flow

SO_REUSEPORT group and it is held using **sock_hold()**. If no socket is found, **inet_csk_reqsk_queue_drop_and_put()** is called to remove expired sockets. The function then calls **tcp_v4_fill_cb()** to maintain all the control buffer states, including selective acknowledgement, TCP sequence number, etc. Next it calls **tcp_v4_do_rcv()** function to check if the TCP connection is already in the established state by verifying the **TCP_ESTABLISHED** state variable. If the connection is already established, **tcp_rcv_established()** is invoked for further processing. **tcp_rcv_established()** function initially opts for fast-path processing by default. It only diverts to slow-path execution if slow-path options are set. Fast path can be disabled for various reasons, such as out of order segments arrival or insufficient buffer space. The **tcp_rcv_established()** function undertakes several operations to decide whether to follow fast path or divert to slow path.

The process begins by checking the timestamp, and if a corrupted timestamp is detected, all the future packets are dropped due to wrapped sequences. If the length of packet is small compared to TCP header length, it discards the packet. Otherwise, verifies the checksum for errors and invokes the **tcp_rcv_rtt_measure_ts()** to measure the round-trip time (RTT) and perform RTT updates. The function then advances the read position of data in socket buffer using **__skb_pull()** function and calls **tcp_queue_rcv()** to enqueue the received data for application usage.

tcp_queue_rcv() function locates the tail of the socket buffer and tries to coa-

lesce the new skb with the previous skb by utilizing `skb_peek_tail()` and `tcp_try_coalesce()` functions. It updates pointers based on next sequence and bytes received by the socket through `tcp_rcv_next_update()`. If not coalesced then queue the complete skb at the tail of skb receive queue by calling `__skb_queue_tail()` function.

Segment processing within the socket's receive queue is complete. When the application invokes the read system call to access data, it subsequently triggers the `tcp_recvmmsg()` function call. The `tcp_recvmmsg()` function is a socket layer function, and we will discuss it in the upcoming section 2.8.

• UDP

`udp_rcv()` handler function is the primary entry function, which directly invokes the other function, `__udp4_lib_rcv()`, to handle the datagram, as shown in Fig. 2.11.

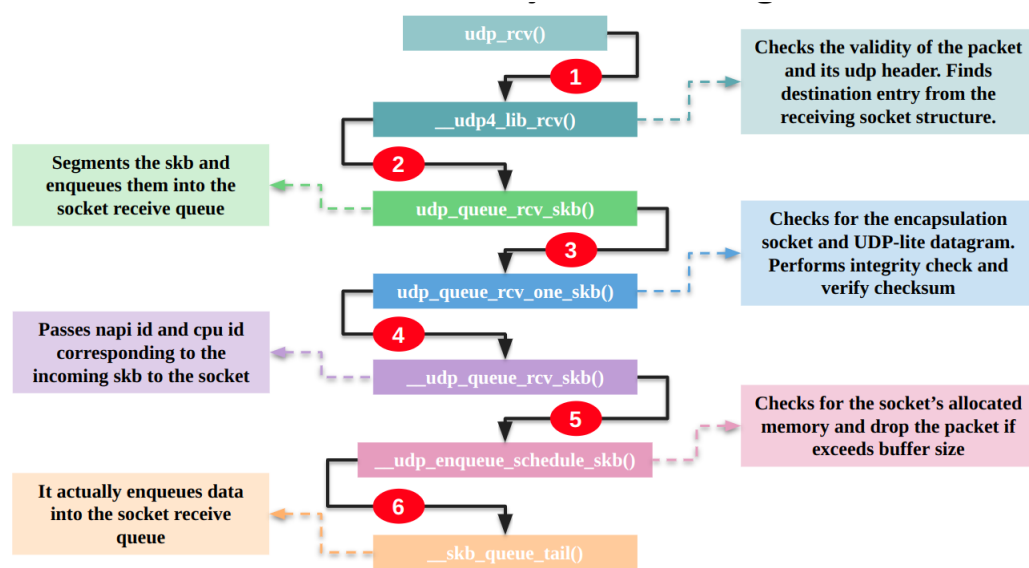


Figure 2.12: UDP Protocol Layer Function Call Flow

In the `__udp4_lib_rcv()`, an initial check on the validity of the datagram and UDP header is performed using `pskb_may_pull()` function. Subsequently, the UDP datagram length, source and destination addresses, and the checksum is verified. The `skb_steal_sock()` function is employed to acquire the socket structure correspondingly find the `dst_entry` using `skb_dst()`. We have seen in previous section that an optimization `early_demux()` is utilized to attach the `dst_entry` to the packet before handing it over to the upper protocol layer. If a `dst_entry` is found,

the datagram is then queued to the respective socket using **udp_unicast_rcv_skb()**. In case, socket not found, **__udp4_lib_lookup_skb()** is called to lookup for receiving socket based on source and destination addresses. In both cases, the data is enqueued to the socket using **udp_queue_rcv_skb()**. The **udp_queue_rcv_skb()** function segments the skb using **udp_rcv_segment()**, and it iterates through all the segments to enqueue them into the socket's receive queue by calling **udp_queue_rcv_one_skb()** function. If, even after calling **__udp4_lib_lookup_skb()**, the socket is not found, the datagram is dropped.

The **udp_queue_rcv_one_skb()** function begins by checking if the socket associated with the datagram is an encapsulation socket. If so, it passes the packet up to the UDP's **encap_rcv()** handler function, otherwise, it is directed to the UDP socket. The function then determines if the datagram is UDP-Lite datagram and performs the necessary integrity check. It verifies the checksum and drops the datagram if checksum error is detected. After all of this, the function calls **__udp_queue_rcv_skb()**. In **__udp_queue_rcv_skb()**, the function passes NAPI id and CPU id corresponding to the incoming skb to the socket. It adds the datagram to the receive queue by calling **__udp_enqueue_schedule_skb()**.

Within **__udp_enqueue_schedule_skb()**, the function checks the socket's allocated memory to determine if the buffer size exceeded. If so, it increments the drop count. It invokes **udp_rmem_schedule()** to ensure that there is sufficient buffer space to accept the datagram. The **__skb_queue_tail()** function is invoked to add the data to the receive queue. Processes waiting for the data to arrive in the socket are notified through the **sk_data_ready()** function call.

This concludes the processing of the UDP protocol layer. In the next section, we will delve into the socket layer data processing.

2.8 Socket Layer Processing

Whenever a user application issues API calls like **read**, **recvfrom**, these calls are mapped onto system calls such as **sys_recv** which gets further translated into the **sys_recvfrom** calls. Subsequently, the **sys_recvfrom** and other **recv** system calls are translated into the **sock_recvmsg()** function. In the case of INET sockets, **inet_recvmsg()** is invoked, lead-

ing to a call to the protocol-specific receive function. Specifically for TCP, `tcp_recvmmsg()` is the function where all system calls are ultimately translated at the socket layer. This function initiates the copying of data from an open socket buffer into a user buffer. The flow from `tcp_recvmmsg()` to socket receive buffer is shown in Fig. 2.13.

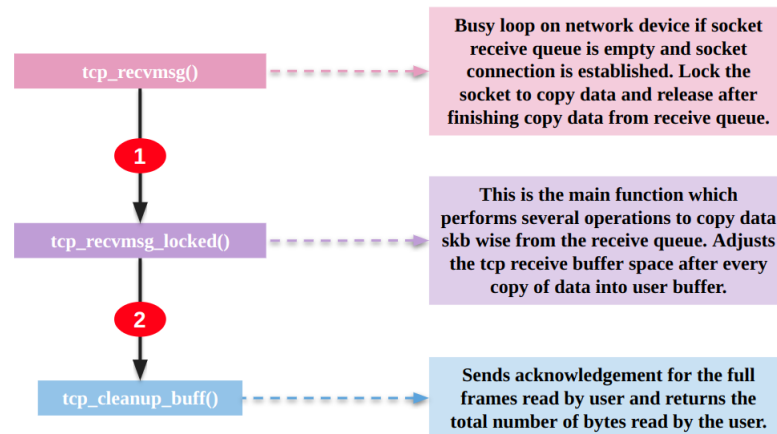


Figure 2.13: Socket Layer Function Call Flow

Within the `tcp_recvmmsg()` function, if the error flag is set, it reads messages from error queue. If socket can busy loop, socket receive queue is empty and `sk_state` is **TCP_ESTABLISHED** then busy loop on network device for some packets. It locks the socket and call `tcp_recvmmsg_locked()` function to copy data from socket receive queue into the user buffer and after completion release the socket. Then call `tcp_recv_timestamp()` to attach timestamp with the message and call `put_cmsg()` to attach some control messages or metadata to the received data for building message header.

`tcp_recvmmsg_locked()` is the main function which is responsible for copying data from the socket's receive queue into the user buffer. It first checks the `sk_state`, if it is **TCP_LISTEN**, an error is returned, as a listen socket only listens the connections. If there are already some bytes in the queue, the `msg_get_inq` field of the message is set for clarity.

The function obtains the socket's receive timeout by using `sock_timeo()`, returning 0 if non-blocking or the timeout value otherwise. If urgent data is received, it is handled without blocking. The function retrieves the head of unread data and stores it in the 'seq' variable. The number of bytes to be read from the socket is determined and stored in the 'target' variable.

The function then enters the loop to copy data from the receive queue, continuing until the user's required number of skbs are copied. Within the loop, it obtains the first skb from the receive queue, finds the offset, and checks if the offset is within the data bounds of the current skb. It calculates the amount of data that can be used starting at the offset in the buffer and stores it in the 'used' variable. The function copies 'used' number of bytes starting at the offset into the user buffer and updates relevant values. It calls **tcp_rcv_space_adjust()** to adjust the TCP receive buffer space after each copy of data into the user buffer. If there is more data, the loop continues, otherwise, it calls **tcp_eat_recv_skb()** to un-link the skb from the socket's receive queue and free the skb. Here the data copy loop ends. At the end, it calls **tcp_cleanup_rbuf()** to send acknowledgment for full frames read by the user and then returns the total number of bytes read by user.

This concludes the processing of the socket layer. In the next section, we will discuss the approaches to programmability in the Linux Kernel Network Stack.

Chapter 3

Current Approaches to Programmability in the Linux Kernel

In this chapter, we will discuss various established methods that offer the scope of introducing programmability into the Linux Kernel. We will begin by discussing the Extended Berkley Packet Filtering (eBPF) and eXpress Data Path (XDP) techniques for kernel customization. Then, we will delve into some of the current state-of-the-art approaches. The first section [3.1](#) discusses ghOSt [\[2\]](#), which focuses on customizing the kernel scheduling decisions by delegating the decision making process for Linux scheduling policies to the user space. In the next section [3.2](#), we will discuss the XRP [\[3\]](#) framework that has been developed to enable applications to carry out user-defined storage operations, like index look-ups or aggregations, through an eBPF hook within the NVMe driver. As kernel software stack is becoming a bottleneck in the performance of new high-speed memory technologies, XRP allows the applications to securely bypass a substantial portion of the kernel's storage stack. At last, we will discuss the Syrup [\[4\]](#), a framework designed to facilitate the user-defined scheduling, similar to ghOSt. Syrup offers application developers the ability to express application-specific scheduling policies without the need to modify the underlying code of system layers. It leverages eBPF and ghOSt as key tools within its framework, to deploy policies at various layers of the kernel stack as described in the upcoming section [3.4](#).

3.1 eBPF and XDP

eBPF (Extended Berkeley Packet Filter) functions as a virtual machine, enabling the extension of kernel functionalities through custom code injection at run-time as described in the eBPF documentation [9]. This code can be executed at various hook points, such as trace points, system calls, or any kernel function. eBPF programs use a special byte-code generated by the Clang/LLVM tool-chain from source code written in a restricted C language. This byte code can be compiled just-in-time into native machine code for enhanced performance.

When injected, eBPF programs undergo verification to ensure they cannot harm the kernel. The verifier checks for allowed memory accesses and guarantees that the program will eventually terminate. Despite some limitations, such as a maximum number of instructions and the absence of support for unbounded loops, eBPF programs have proven suitable for creating reasonably complex Network Functions (NFs), particularly those focused on header processing.

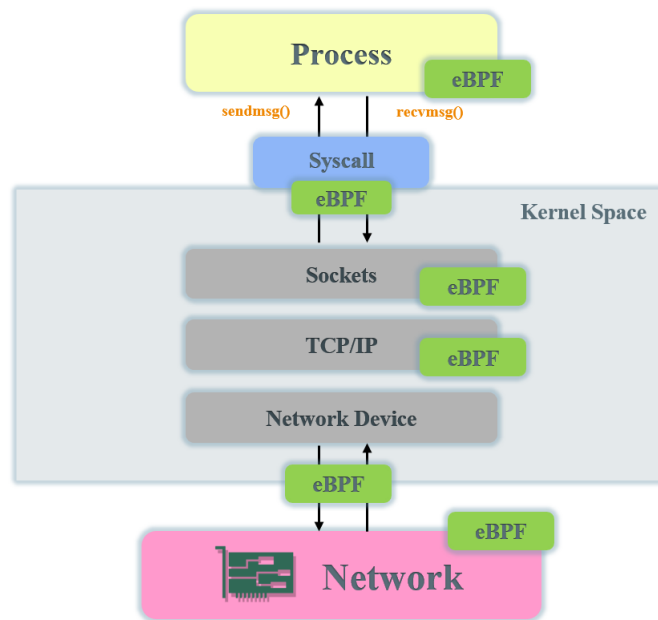


Figure 3.1: eBPF Architecture

The eXpress Data Path (XDP) provides a hook for executing high-speed eBPF packet processing programs before entering the main Linux kernel. In its native mode, XDP allows these programs to run in the Network Interface Card (NIC) driver, at the earliest

possible point after receiving a packet from the hardware. This occurs before the kernel allocates its per-packet `sk_buff` data structure or performs any parsing. Depending on the processing result, the program can instruct the kernel to drop the packet, allow it to continue for further processing in the network stack, send it back on the receiving interface, or redirect it to another network device or user space using AF_XDP sockets.

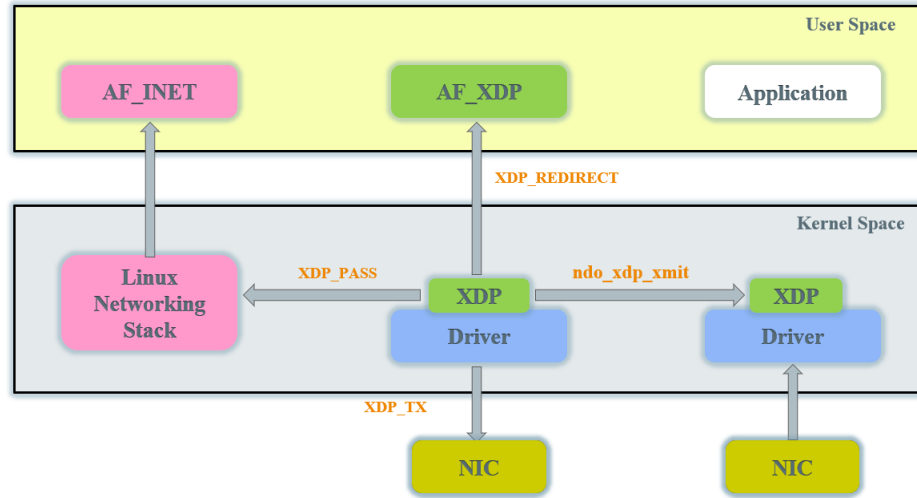


Figure 3.2: XDP Architecture

3.2 ghOSt: Fast and Flexible User-Space Delegation of Linux Scheduling

ghOSt [2] provides a framework to offer versatile user-space offloading of scheduling policies within a Linux environment. Implementing, testing, and efficiently deploying new kernel schedulers poses considerable challenges, especially when dealing with a large number of systems. But, ghOSt facilitates the development and optimization of policies in any programming language, permitting modifications without the need for rebooting host. CPU scheduling is a crucial factor influencing both application performance and security. Customizing policies to suit specific workload types can significantly enhance critical metrics, including latency, throughput, real-time characteristics (both hard and soft), energy efficiency, cache interference, and overall security.

The ghOSt framework incorporates an abstraction and interface that empowers user space

software to design intricate scheduling policies, ranging from per-CPU models to system-wide (centralized) models. The ghOSt separates the kernel scheduling mechanism from the policy definition. The mechanism lies in the kernel, which does not change frequently and policy definition lies in user space, which changes frequently according to user application requirements. Let's discuss the design of ghOSt framework.

ghOSt Design

In the ghOSt architecture as shown in Fig. 3.3, scheduling policy logic operates within regular Linux processes, identified as 'agents', which communicate with the kernel through the ghOSt API. Usage of native threads for scheduling helps in supporting existing applications also without any modifications need. The agents can be coded in any programming language to make it easier for users to implement and test. They made this design fault-tolerant and isolation-supportive by allowing the system to fall back to the default scheduler if any agent crashes.

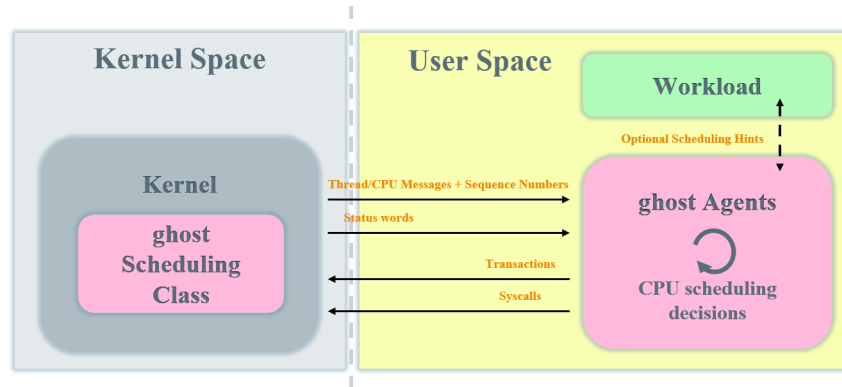


Figure 3.3: ghOSt Architecture

In case of per-CPU scheduling model, one agent thread is allocated to each core in the system. Each agent implements and handles thread scheduling of its own CPU, as shown in Fig. 3.4. Each agent also maintains run queues, task state, etc.

The kernel asynchronously transmits generated messages about task state when an event occurs, like thread creation, thread block, etc., to the user space agent via shared memory message queues. Subsequently, the agent communicates the scheduling decisions to the kernel via transaction commits, synchronously. These transactions are atomic and retractable to make design more flexible. An example of per-CPU agent code is shown in Fig. 3.5.

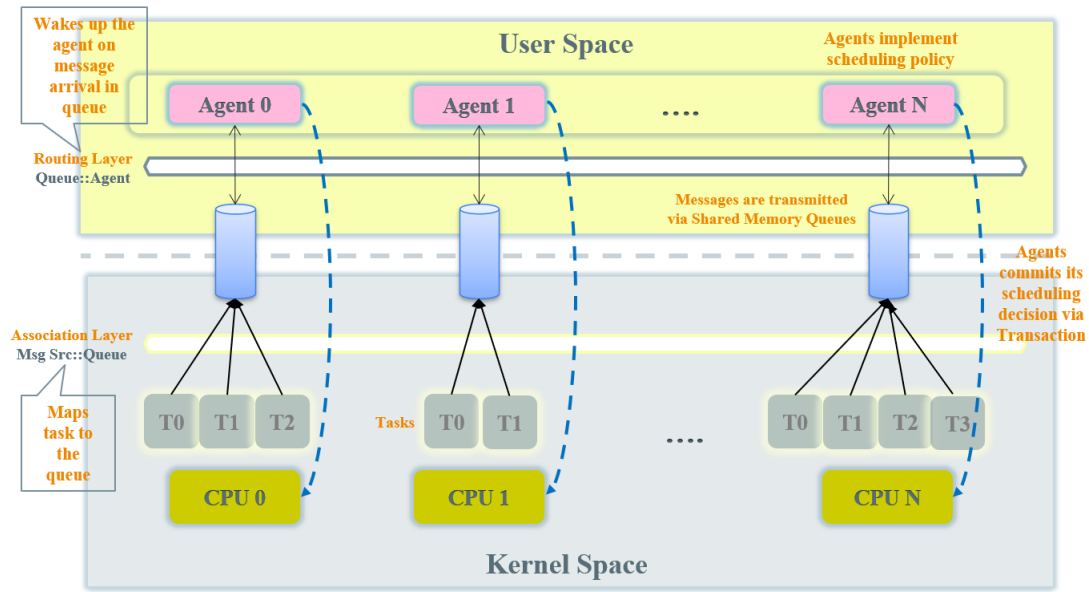


Figure 3.4: ghOST: per-CPU Scheduling Design

```

1 void Agent :: PerCpuSchedule ()
2 {
3     DrainMessageQueue(); // Read messages from queue
4     Thread *next = runqueue_.Dequeue();
5     if(next == nullptr)
6         return ; // Runqueue empty.
7     // Schedule thread :
8     Transaction *txn = TXN_CREATE(next->tid, my_cpu);
9     TXNS_COMMIT({txn});
10    if(txn->status != TXN_COMMITTED)
11    {
12        // Txn failed. Move thread to end of runqueue.
13        runqueue_.Enqueue(next);
14        return;
15    }
16    // The schedule has succeeded for 'next '.
17 }
18

```

Figure 3.5: ghOST: Thread Scheduling code in per-CPU Scheduling Design

While in the case of centralized scheduling, a single global agent implements and handles the scheduling decisions for all the CPUs and all other satellite agents remain inactive, as shown in Fig. 3.6. The communication between user space agent and kernel occurs same as per-CPU model via asynchronous messages and synchronous transaction commits.

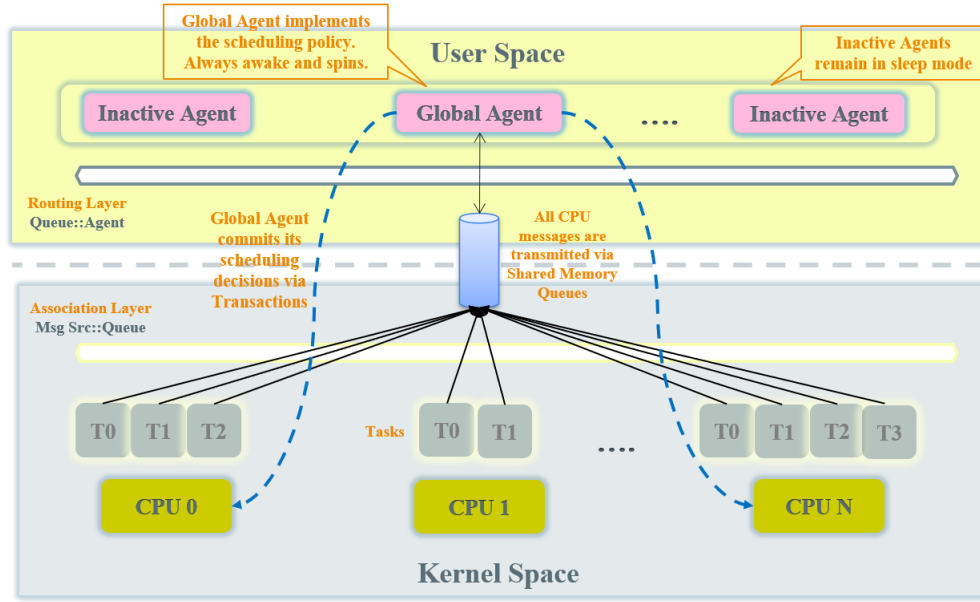


Figure 3.6: ghOSt: Centralized Scheduling Design

3.3 XRP: In-Kernel Storage Functions with eBPF

As shown in Fig. 3.7, the data path for a typical read request on Optane SSD Gen2, consists of the application itself, the kernel boundary, the syscall layer, the file system and block layer, the NVMe driver, and finally, the storage device. Within this path, 50% of the read latency is attributed to the traversal of the kernel storage stack, encompassing the journey of the read query from the kernel boundary up to the NVMe driver.

XRP addresses this challenge by bypassing the kernel software and offloading an eBPF function to the NVMe driver, thereby preventing queries from traversing this kernel segment multiple times. For instance, while executing an index lookup query, traditional methods involve traversing the kernel storage stack multiple times to perform the dependent read requests chain, as shown in Fig. 3.8. Dependent read requests chain is a general action, performed in most of the storage engines, like B-Tree and Long Structured Merge (LSM) Tree.

But using XRP, it has to traverse the kernel software only twice as shown in Fig. 3.9, first while initiating the chain of read requests and another while retrieving the final response of read query from the storage device to the user space application. XRP facilitates the acceleration of various operations, including index look-ups, aggregations, and range

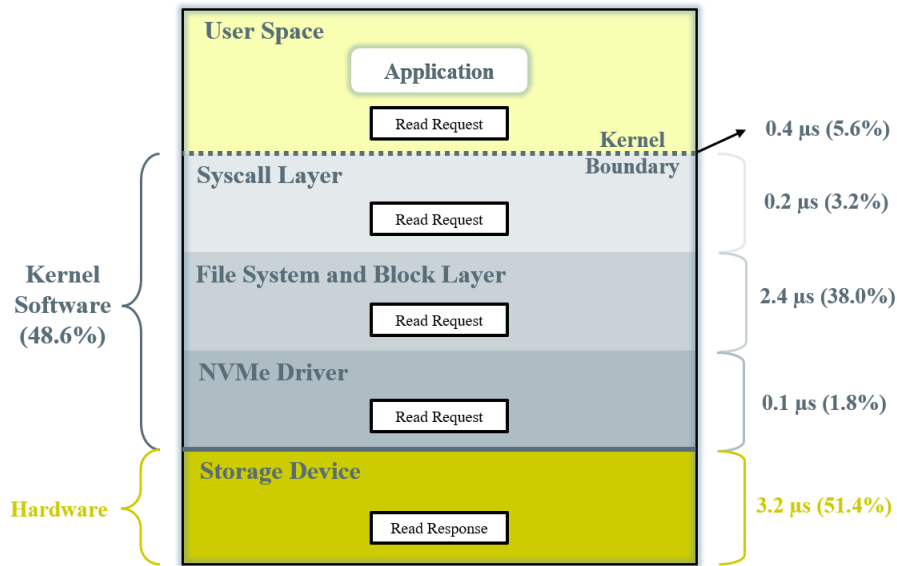


Figure 3.7: Kernel Storage Stack Layer-wise Read Request Latency

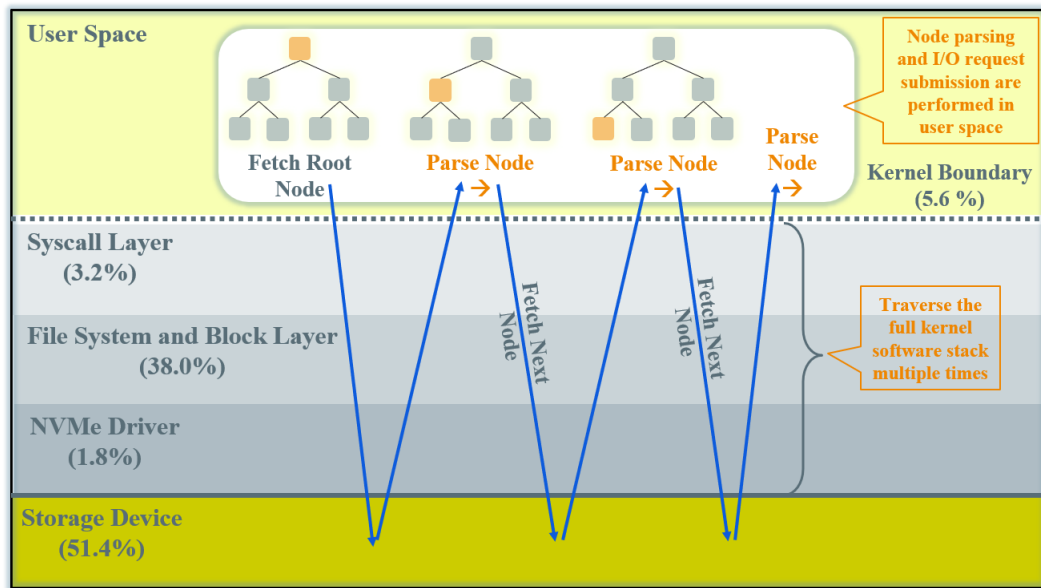


Figure 3.8: B+ Tree Index Lookup Process From User Space

queries. Let's explore the details of XRP framework design.

XRP Design: The XRP architecture is shown in Fig. 3.10, comprises a BPF hook function that incorporates query re-submission logic, strategically placed within the NVMe interrupt handler. Additionally, the architecture involves a file system translation step,

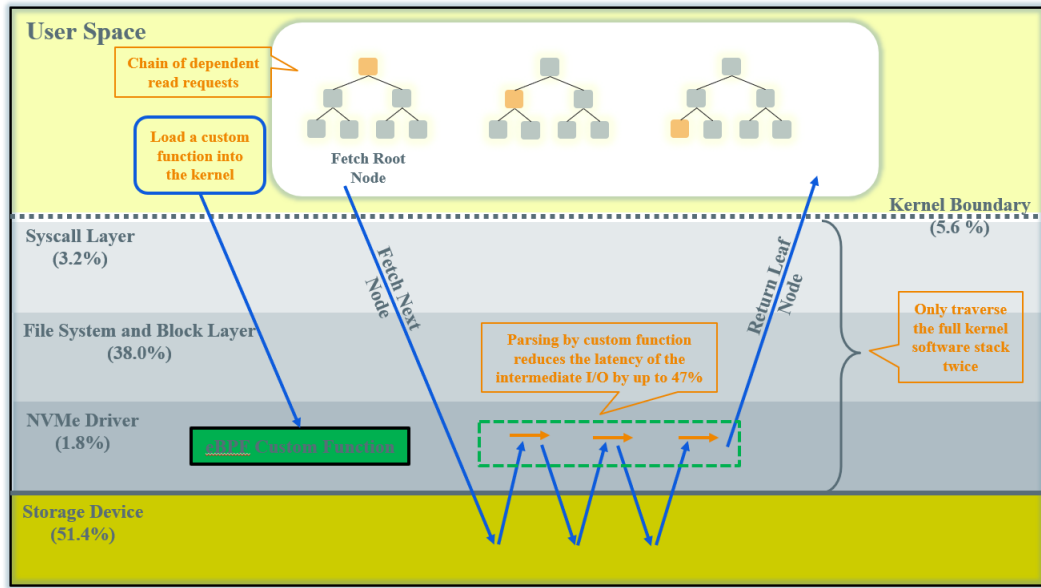


Figure 3.9: XRP: A Framework for In-Kernel Storage Functions

and the construction and re-submission of the query at the device.

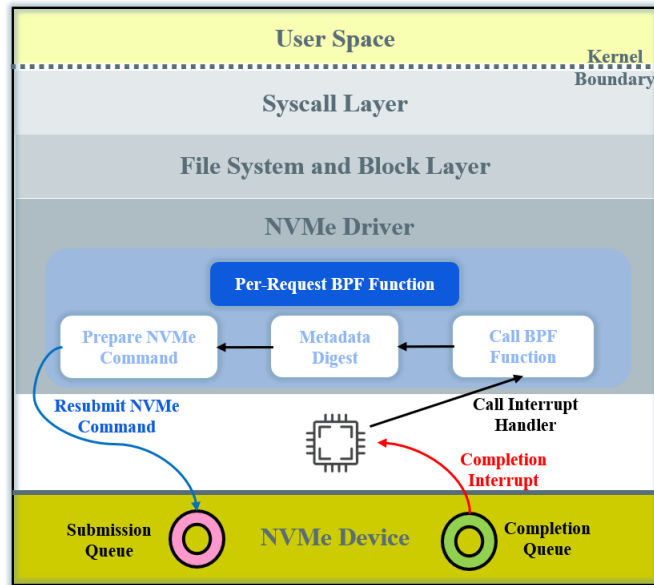


Figure 3.10: XRP Architecture

Upon completion of NVMe request on the storage device, the kernel is interrupted to trigger the interrupt handler. In the interrupt context, XRP calls its corresponding BPF function with the next address to access after each NVMe request completion. To de-

termine the next logical address translation to physical address, it invokes the metadata digest consisting of a thin interface between the file system and the interrupt handler, facilitating the sharing of logical to physical block mappings with the interrupt handler. This mechanism also enables the secure eBPF based on-disk re-submissions.

During re-submission, XRP readies the NVMe re-submission command and adds it to the NVMe submission queue (SQ) of the respective core. Following the completion of the request, a completion interrupt is generated by the device, prompting the interrupt handler to call the corresponding BPF function. The BPF function assesses whether to resubmit the query or return the response to the user space application.

The signature of BPF programs that XRP can load is shown in Fig 3.11, featuring fields designated for BPF function use only and fields that can be accessed outside the BPF context. XRP uses two buffers: the 'data buffer' stores data read from the disk, while the 'scratch buffer' serves as private space for both the user and the BPF function to pass the parameters, like search key. An example of BPF function for key-value storage engine (BPF-KV) is shown in Fig. 3.11, encompassing re-submission code and the node structure.

The primary challenge encountered involved the translation of next logical address to physical address within the interrupt handler. Given that the interrupt handler lacks the awareness of logical to physical address mappings and the translation requires some file system meta-data access. To resolve this issue, they implemented a metadata digest comprising two functions: an 'update function' callable within the file system, triggered when the file system updates the logical to physical address mappings, and a 'lookup function' callable within the interrupt handler, providing the necessary mappings. Importantly, these two functions are designed particular to each file system.

3.4 Syrup: User-Defined Scheduling Across the Stack

To illustrate the need for user-defined scheduling throughout the kernel stack, Syrup investigated a RocksDB workload scenario. In this example, two scheduling policies, Vanilla Linux and Round Robin (implemented in Syrup), were employed to handle a homogeneous GET requests workload. As depicted in Fig. 3.12, Round Robin demonstrates a 99% latency achievement at very high traffic loads compared to Vanilla Linux, with a

```

1 struct bpf_xrp {
2     // Fields inspected outside BPF
3     char *data;
4     int done;
5     uint64_t next_addr[16];
6     uint64_t size[16];
7     // Field for BPF function use only
8     char *scratch;
9 };
10 uint32_t BPF_PROG_TYPE_XRP(struct bpf_xrp *ctxt);
11
12 struct node {
13     uint64_t num; uint64_t type;
14     uint64_t key[31]; uint64_t ptr[31];
15 };
16
17 uint32_t bpfkv_bpf(struct bpf_xrp *ctxt) {
18     uint64_t key = *((uint64_t*)ctxt->scratch);
19     struct node *n = (struct node *)ctxt->data;
20     if (n->type == LEAF_NODE)
21     {
22         ctxt->done = true;
23         return 0;
24     }
25     int i;
26     for (i = 1; i < n->num; i++)
27         if (key < n->key[i]) break;
28     ctxt->done = false;
29     ctxt->next_addr[0] = n->ptr[i - 1];
30     ctxt->size[0] = 512;
31     return 0;
32 }

```

Figure 3.11: A BPF program signature and BPF function for BPF-KV

significant reduction in the percentage of request drops. This example shows that Vanilla Linux may not be the optimal scheduling policy for all types of applications and workloads. Matching the perfect scheduling policy to a particular type of workload can address issues such as load imbalance and head-of-line blocking. Application-specific scheduling policies, as demonstrated in the RocksDB server example, have the potential to enhance performance significantly.

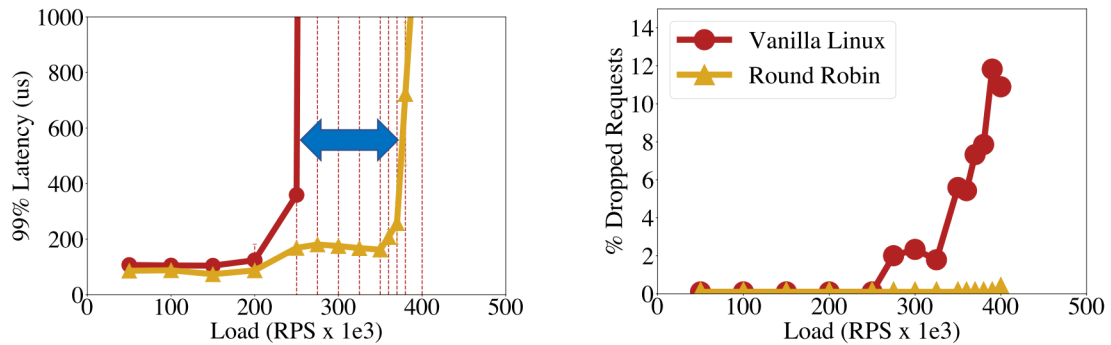


Figure 3.12: RocksDB benchmark performance with 100% GET requests

Syrup successfully meets various requirements for providing user-defined scheduling, including expressibility, cross-layer deployment, low overhead and isolation. Expressibility in Syrup offers the flexibility to select the optimal scheduling policy for different applications and workloads to enhance performance. For example, First-Come-First-Serve (FCFS) scheduling policy is suitable for workloads having low variability, while preemption based or resource partitioning scheduling policies could be more effective for high variability workloads. Syrup achieves expressibility by translating scheduling policies into online matching functions. Additionally, Syrup's scheduling capability extends across multiple layers of the stack. Implementing scheduling policy at right layer, improves the performance. For example, sometimes it is beneficial to offload packet scheduling to hardware to improve scalability. Syrup leverages eBPF and ghOSt techniques to implement policies at different layers of the stack, offering adaptability to diverse workloads. With many modern workloads operating at the microsecond scale, Syrup ensures that new scheduling policies can be implemented and deployed without imposing excessive overhead on the system. In the context of multi-tenancy, where multiple hosts run on the same machine, Syrup ensures isolation by employing a global arbiter for multiple applications. Now, let's delve into the design of Syrup.

Syrup Design:

Fig. 3.13 illustrates the key components of the Syrup scheduling workflow, with the functions of numbered arrows described as follows:

1. The application developer specifies the scheduling policy in a file using the eBPF interface C language. It includes the details about the desired scheduling policy, target hook points and the key-value map.
2. The application invokes the Syrup daemon (Syrupd) to deploy the specific scheduling policy outlined in the application code.
3. Syrup daemon compiles the application policy file into a binary file.
4. Syrupd deploys the scheduling policy at various target hooks specified by the application.
5. Both the application and the scheduling policy, deployed by Syrupd at different layers of the system, passes information such as load, latency, completion time, etc., utilizing a key-value map abstraction stored in application policy file.

This explains how the Syrup allows user applications to interact with the system using scheduling policies.

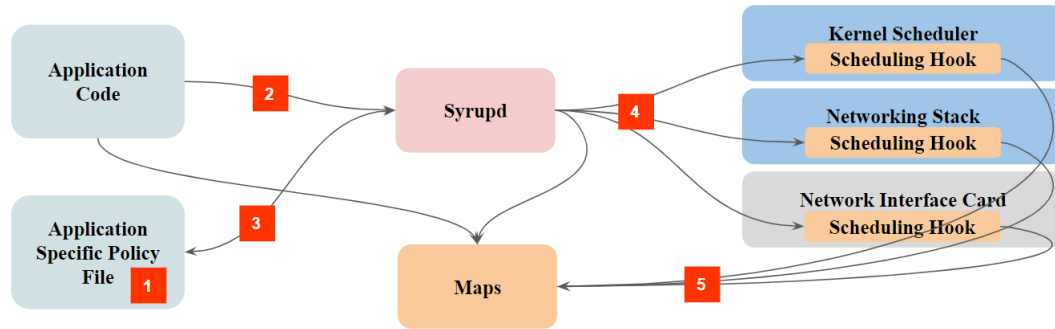


Figure 3.13: Syrup Scheduling Workflow

Syrup utilizes ghOSt, eBPF hardware and eBPF software back-ends to implement multiple hooks. **Within the eBPF software back-end, various hook points are available throughout the kernel networking stack, enabling secure deployment of scheduling policies.** Syrup leverages eBPF hardware to apply policies at programmable network devices, specifically SmartNICs. Additionally, ghOSt is employed by Syrup to take advantage of its features for offloading thread scheduling to the user space.

As described above, Syrup formulates scheduling policies as match functions that take inputs such as connections, network packets, and threads, and outputs the executors, such as network sockets, NIC queues, cores. Fig. 3.14 represents Syrup hooks for scheduling across the stack, incorporating both eBPF and ghOSt.

The deployment of user-defined scheduling policies across multiple layers of the kernel is facilitated through eBPF hook functions. The **Thread Scheduler hook** function allows users to define scheduling policies that take thread as input and determines the core on which it should run. The **Socket Select hook** directs multiple TCP and UDP connections to the designated TCP and UDP sockets on the same port, especially in the case of the `SO_REUSEPORT` option. The **CPU Redirect hook** empowers the users to steer the network packets to the desired core for processing within the kernel network stack. The **XDP_SKB hook** and **XDP_DRV hook** take network packets as inputs and are handled by the `AF_XDP` network socket, diverting packets directly to the user space, by-passing the kernel before reaching the protocol processing layer. The **XDP offload hook** provides users with the ability to offload scheduling policies onto the network device and direct the packets to the user-specified NIC RX queues. This outlines how the Syrup

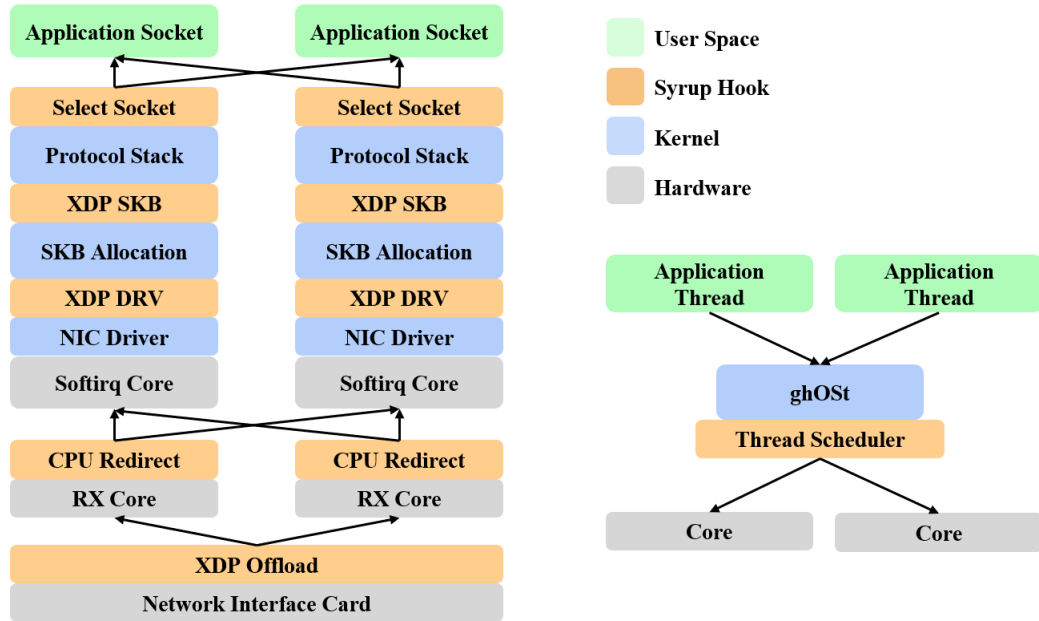


Figure 3.14: Syrup-supported hooks across the Linux Stack

enables user-defined scheduling across different layers of the kernel stack.

3.5 Summary

ghOSt serves as an illustrative example, showcasing the programmability of the kernel at the CPU level. If we talk about the contemporary Linux network stack, NAPI is a crucial component employed to mitigate interrupts and prevent packet drops, especially under high traffic loads. NAPI achieves this by delegating the packet processing task to the ksoftirqd process, which continuously runs in a loop on the CPU. Consequently, the network stack's performance is intricately linked to the scheduling policy. For applications prioritizing low latency and network operations tailored for low-latency requirements, the scheduling policy emerges as a critical factor. There is potential for exploring the integration of ghOSt alongside NAPI to augment and tailor the scheduling configuration within the networking domain of the kernel. This opens up opportunities for customized scheduling configurations specifically designed to meet networking requirements.

The XRP framework provides a concrete demonstration of the possibilities enabled by kernel programmability support, empowering users to enhance storage performance in

response to specific requests.

Similarly, Syrup facilitates user-defined scheduling across multiple layers of the kernel stack, encompassing thread scheduling, network packet scheduling, and connections redirection to sockets through various eBPF hooks integrated into the kernel stack.

Collectively, these initiatives pave the way for exploring the development of a customized Linux network stack that users can configure based on their performance or service requirements. This promising research direction will be elaborated upon in the concluding section dedicated to prospective research directions.

Chapter 4

Use Cases Demanding Enhanced Programmability in Linux Network Stack

In this chapter, we will explore various scenarios that highlight the necessity for programmability in the Linux network stack. The ability to customize the network stack's operations is vital for aligning its functionality with user-specific requirements. Let's delve into some of the possibilities that could be incorporated into the kernel network stack, extending its capabilities to user-space applications.

Custom Load Balancing Logic: We can offload the load balancing decisions to user space, enabling the implementation of custom algorithms for dynamic adjustments based on specific application requirements. For instance, implement a user-defined load balancing algorithm in user space to distribute traffic based on criteria like session affinity or resource utilization. This is valuable for applications such as a Content Delivery Network (CDN) with a custom load balancing algorithm prioritizing edge servers based on geographic proximity and server load.

Customized Packet Processing: We can develop eBPF programs for packet processing tasks and offload them to user space to enable highly customized and flexible packet handling outside the kernel. This approach supports specialized application firewalls with tailored deep packet inspection logic and allows for user-defined packet filtering

rules, making it ideal for applications like Intrusion Detection Systems (IDS).

Dynamic Buffer Allocation: We can develop user-defined policies for dynamic allocation of buffer sizes to RX and TX ring buffers, based on the traffic patterns, quality of service (QoS), and available system resources. This optimization can be implemented across all applications to ensure efficient memory usage.

Policy-Based Routing: We can offload the policy-based routing decisions to user space for dynamic adjustments based on factors such as network conditions, application requirements, or security policies. This enables real-time changes to routing policies without requiring kernel reconfiguration, making it suitable for multi-homed applications adjusting routing based on metrics or fail over conditions.

4.1 Enabling and Implementing the Enhanced Programmability

All the above mentioned customized functionalities can be enabled in the network stack using three methods:

1. The first approach involves securely providing the required access to physical resources and kernel functions for user-space applications. This is accomplished by making all the necessary modifications to the kernel and deploying it specifically for the targeted applications.
2. The second method entails defining new eBPF or XDP hook points based on the functionalities required by user applications. An API is provided to users, and user-defined functions are added to these new hook points.
3. The third method involves leveraging existing eBPF and XDP hook points with some kernel modifications. User-defined code is developed to offer all the desired user-specific functionalities while utilizing the features provided by the existing hooks.

The first method, as described in the previous section for enabling the enhanced programmability, is a time-consuming and complex process that requires expertise in the kernel programming. Achieving the desired programmability goal using this method is

challenging. Moreover, it involves submitting the customized kernel to the Linux community, waiting for approval, and then relying on the next kernel release to incorporate all the changes. This makes it a time-consuming process, limiting the immediate usability of the targeted applications.

The second method involves the complex task of writing and deploying new hooks in the kernel network stack. While it is a tedious task, it is less complex compared to the first method.

The third method is the easiest way to introduce programmability into the kernel network stack. User application developers can articulate their requirements using a standard programming interface and deploy them through the existing hooks, providing a more straightforward and accessible approach to achieve programmability.

Chapter 5

Prospective Research Directions for PhD

In the previous section, we explored various use cases that demand the customization of the Linux network stack, along with the methods for enabling and implementing enhanced programmability. However, designing customized kernel network stack for each type of use cases is not an efficient way to provide a programmability solution.

An alternative approach is to develop a patched kernel network stack that includes hook points for deploying all user-defined functionalities, with the added capability of enabling or disabling these features. This approach allows users to reconfigure the kernel network stack based on their application requirements. Users have the flexibility to decide whether to enable or disable specific hook functions. For instance, an eBPF hook for Generic Receive Offloading (GRO) could be provided, and users can choose whether to utilize this feature. Some applications may not require coalescing packets, as their requests are small enough to fit in a single packet or a few packets. In such cases, passing each packet through GRO processing incurs unnecessary overhead. In the current kernel network stack, GRO is typically either enabled or disabled for all applications, with no individual application-level control over whether to undergo GRO processing. This limitation highlights the need for more fine-grained control and configurability in the Linux network stack.

Moreover, consider another example of deciding the logic of the optimization functions utilized for the packet processing. In the existing GRO logic, the process awaits the

accumulation of a predetermined maximum number of similar packets to merge them into a single packet. However, this approach may not be optimized for applications with microsecond-level performance requirements. Alternatively, users could have the ability to impose a time constraint on packet reception in the GRO list. After a user-defined time duration, the system would then merge the available similar packets into a single packet. This illustrates the broader concept of empowering users to determine the most suitable functions for their application's performance or service requirements.

The conceptualization of the patched kernel network stack is illustrated through an example, shown in Fig. 5.1. The diagram depicts a customizable chain of functionalities that users can selectively enable or disable using function enablers or disablers. These user-driven decisions are then implemented in the kernel network stack through the deployment of hook functions on designated hook points. Users have the flexibility to configure their requirements by choosing from a range of functions, including packet queuing, GRO processing, RPS processing, packet filtering, and more.

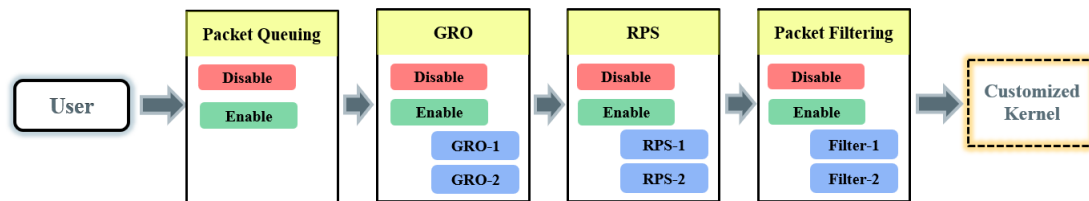


Figure 5.1: Selective Chain of Functions

Presently, we can commence the further research by utilizing the existing hooks and defining eBPF functions to implement user-defined requirements within the kernel network stack.

Bibliography

- [1] D. R. Engler, M. F. Kaashoek, and J. O'Toole. Exokernel: an operating system architecture for application-level resource management. In *Fifteenth ACM symposium on Operating systems principles (SOSP '95)*, pages 251–266, 1995.
- [2] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. Ghost: Fast & flexible user-space delegation of linux scheduling. In *ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*, page 588–604, 2021.
- [3] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. XRP: In-Kernel storage functions with eBPF. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, Carlsbad, CA, July 2022. USENIX Association.
- [4] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. Syrup: User-defined scheduling across the stack. In *ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*, page 605–620, 2021.
- [5] Monitoring and tuning the linux networking stack: Receiving data, (accessed November 17, 2023). <https://blog.packagecloud.io/monitoring-tuning-linux-networking-stack-receiving-data/>.
- [6] Illustrated guide to monitoring and tuning the linux networking stack: Receiving data, (accessed November 17, 2023). <https://blog.packagecloud.io/illustrated-guide-monitoring-tuning-linux-networking-stack-receiving-data/>.

-
- [7] The linux kernel documentation, (accessed November 17, 2023). <https://docs.kernel.org/>.
 - [8] Scaling in the linux networking stack, (accessed November 17, 2023). <https://www.kernel.org/doc/Documentation/networking/scaling.txt>.
 - [9] ebpf documentation, (accessed November 17, 2023). <https://ebpf.io/what-is-ebpf/>.